

เอกสารแผนการทดสอบระบบ

PJChatbot v2 — RAG Welfare Assistant

ชื่อระบบ:	ระบบทดสอบแชทบอทข้อมูลสวัสดิการ รพ.ธรรมศาสตร์เฉลิมพระเกียรติ
จัดทำโดย:	ทีมวิศวกรรมทดสอบระบบ (Functional QA, Performance QA, AI Engineer)
ระดับความปลอดภัย:	ข้อมูลภายในสถาบัน (Confidential)
วันที่จัดทำแผน:	20 สิงหาคม 2569
สถานะแผน:	อนุมัติแผนงานการทดสอบ (Approved)

1. วัตถุประสงค์และขอบเขตการทดสอบ (Objectives & Scope)

แผนการทดสอบฉบับนี้จัดทำขึ้นโดยความร่วมมือของทีมทดสอบระบบ PJChatbot เพื่อกำหนดทิศทาง ขอบเขต และเกณฑ์มาตรฐานในการประเมินความถูกต้องด้านฟังก์ชันงาน (Functional), ประสิทธิภาพการรับโหลด (Performance), ความปลอดภัย (Security) และความแม่นยำในการให้ข้อมูลอ้างอิงของปัญญาประดิษฐ์ (AI RAG Metrics)

1.1 วัตถุประสงค์ (Objectives)

- เพื่อตรวจสอบว่าฟังก์ชันการทำงานทุกส่วนในหน้าแชทบอทของบุคลากรและระบบบริหารจัดการ PDF/ประกาศ หลังบ้านทำงานได้ถูกต้อง 100%
- เพื่อวัดระดับความมั่นคงของแอปพลิเคชันหลังได้รับการแก้ไข CORS Policy และการป้องกัน Path Traversal ในระดับ API Endpoint
- เพื่อประเมินความสั่นไหวและอัตราเวลาตอบสนองเมื่อมีบุคลากรยิงแชทถามคำถามพร้อมกันปริมาณมาก
- เพื่อลดความผิดพลาดในการส่งคืนข้อมูลความรู้ (Hallucination) และตรวจสอบความแม่นยำของ Semantic Vector Retrieval

1.2 ขอบเขตระบบที่รับการทดสอบ (Test Scope)

- ระบบฝั่งผู้ใช้ทั่วไป (Client App): ทดสอบฟังก์ชันการถาม-ตอบ, การแสดงผลลิงก์เอกสารดาวน์โหลด, หน้าเมนูประกาศสไลด์แสดงข่าว และปุ่มกดให้คะแนนความพึงพอใจ (Like/Dislike)
- ระบบบริหารจัดการแอดมิน (Admin Portal): ทดสอบฟังก์ชัน CRUD การอัปโหลดเอกสาร PDF, การจัดการแบบฟอร์มสวัสดิการ, การบันทึกและลบประกาศข่าวสาร, การตรวจสอบหน้าบันทึกประวัติการตอบและหน้าดูคำถามค้างตอบ
- บริการ API หลังบ้าน (FastAPI Endpoints): ทดสอบระบบความปลอดภัย CORS, อัตรา Rate-limiting, การเข้ารหัสผ่าน JWT Authentication, และฟังก์ชันจำกัดโพลเดอร์ดาวน์โหลด (Path Traversal Protection)

2. รูปแบบการทดสอบและเครื่องมือที่ต้องใช้ (Methodologies & Tools)

กระบวนการทดสอบจะถูกแบ่งออกเป็น 4 มิติหลัก โดยมีรายละเอียดกลยุทธ์และชุดเครื่องมือที่จำเป็นต้องนำมาใช้ปฏิบัติงานดังนี้:

2.1 Functional UI & Integration Testing (ผู้ดูแลรับผิดชอบ: เทสเตอร์คนแรก)

- รูปแบบ: ทดสอบการตอบสนองของ UI, พฤติกรรมการสลับเมนูแท็บ และการเชื่อมต่อข้อ API ในหน้าจอจริง

- เครื่องมือที่ต้องใช้:

1. Playwright (Node.js/Python): ใช้เขียนสคริปต์ทำ End-to-End Automated Testing สำหรับกรอกฟอร์มล็อกอิน, กดยกเลิกสิทธิ์หน้าเอกสาร PDF และบันทึกฟิลด์แคช
2. Postman หรือ Bruno: ใช้สร้างชุดการทดสอบ API Integration Test ตรวจสอบเช็คสถานะ HTTP Response (2xx/4xx/5xx)
3. GitHub Action: สำหรับรันสคริปต์เทสอัตโนมัติทุกครั้งเมื่อมีการ push โค้ดใหม่ขึ้นคลาวด์

2.2 Performance & Load Testing (ผู้ดูแลรับผิดชอบ: เทสเตอร์คนสอง)

- รูปแบบ: ทดสอบจำลองปริมาณผู้ใช้เซทบทพร้อมกันเพื่อวัดอัตรา Latency และคอขวดบน CPU/RAM ของเซิร์ฟเวอร์

- เครื่องมือที่ต้องใช้:

1. k6 (JavaScript-based Load Generator): ใช้สร้างโหลดจำลองผู้ใช้งานเซททดสอบถามคำถามพร้อมกันตั้งแต่ 50, 100, จนถึง 500 Virtual Users (VUs)
2. Locust (Python): เครื่องมือสร้างโหลดสำรองเพื่อวัดอัตราความสำเร็จของ RESTful Endpoint หลังบ้าน
3. PM2 / FastAPI Stats Monitoring: ตรวจสอบวัด Resource consumption บน OS ฝั่ง Windows Server

2.3 Usability & Compatibility Testing (ผู้ดูแลรับผิดชอบ: ทดสอบคนสอง)

- รูปแบบ: ตรวจสอบความถูกต้องและสัดส่วนหน้าจอการเรนเดอร์ในอุปกรณ์ที่ต่างกัน ตลอดจนความลื่นไหลในการแสดงผลภาษาไทย
- เครื่องมือที่ต้องใช้:
 1. BrowserStack: จำลองการเปิดเว็บแชทและแดชบอร์ดบนสมาร์ตโฟนระบบ iOS, Android, แท็บเล็ต และหลากหลายเว็บเบราว์เซอร์
 2. Lighthouse / Chrome DevTools: ประเมินคะแนน UX, ความรวดเร็วในการเปิดหน้าเว็บครั้งแรก (LCP) และสถิติ Core Web Vitals

2.4 AI RAG & Data Quality Testing (ผู้ดูแลรับผิดชอบ: วิศวกร AI และข้อมูล)

- รูปแบบ: วัดความแม่นยำของ Retrieval Database (Chroma DB/BM25) และสถิติเวลาการสร้างคำตอบ (LLM Latency)
- เครื่องมือที่ต้องใช้:
 1. Ragas (Retrieval Augmented Generation Assessment): เครื่องมือวิเคราะห์ RAG Pipeline เพื่อวัดผลคะแนนความตรงประเด็นของคำตอบ (Faithfulness, Answer Relevance) และความคุ้มค่าของการดึงข้อมูลความรู้ (Context Recall, Context Precision)
 2. TruLens หรือ DeepEval: ใช้ตรวจวัดสถิติดักจับคำตอบมโน (Hallucination Tracking) และความเหมาะสมของเนื้อหาข้อความตอบกลับภาษาไทย
 3. Custom Python CLI & MLflow: ระบบจัดทำชุดคำถามมาตรฐานสีทอง (Gold Standard Q&A Dataset) ในรูปแบบไฟล์ JSON เพื่อยิงทดสอบระบบ RAG ซ้ำเป็นชุดทดสอบบิดเบี่ยงความคลาดเคลื่อน

3. เกณฑ์ผ่านการทดสอบและแผนงาน (Criteria & Schedule)

3.1 เกณฑ์การผ่านและยอมรับระบบ (Test Pass/Fail Criteria)

- ระดับความรุนแรงของข้อผิดพลาด (Severity Level): จะต้องไม่มีข้อผิดพลาดระดับ Critical หรือ Major (เช่น หน้าเว็บค้างล็อกอินไม่ได้, ฐานข้อมูลขัดข้อง, หรือมีช่องโหว่ความปลอดภัยระดับร้ายแรงเหลืออยู่)
- อัตราความสำเร็จของคำขอ (API Success Rate): อัตราความสำเร็จในการดาวน์โหลดแบบฟอร์มสวัสดิการและดาวน์โหลดข้อมูลประวัติการแพทย์ต้องไม่น้อยกว่า 99.9% ในสภาวะการทำงานปกติ
- อัตราความรวดเร็วในการตอบสนอง (Response Latency): อัตรา Latency ของ API ทั่วไปต้องต่ำกว่า 100ms และสำหรับ API แชนแนล RAG (ที่ต้องรอ AI สรุปความรู้) ต้องแสดงเวลา Token แรกเริ่มปรากฏเฉลี่ยต่ำกว่า 3 วินาที
- ดัชนีความถูกต้องปัญญาประดิษฐ์ (AI Faithfulness Score): คะแนนประเมินของ Ragas ด้านความถูกต้องอิงตามความเป็นจริงในหน้าเอกสาร PDF ต้องมีค่ามากกว่า 0.85

3.2 แผนงานและกรอบเวลาการดำเนินงาน (Execution Timeline)

- สัปดาห์ที่ 1 - Preparation Phase: จัดเตรียมชุดข้อมูลทดสอบมาตรฐาน (Gold Dataset) และเขียน Automated Test cases ใน Playwright
- สัปดาห์ที่ 2 - Execution Phase 1 (Functional & API): เริ่มรันสคริปต์ทดสอบหน้าบ้าน-หลังบ้าน และรวบรวมฟีดแบ็กการแสดงผลลิงก์ข้ามหน้า PDF
- สัปดาห์ที่ 3 - Execution Phase 2 (Load & AI RAG): ดำเนินการยิงโหลดทดสอบด้วย k6 และนำค่า Log ล่าสุดไปคำนวณวัดผลด้วยโมเดล Ragas เพื่อตรวจสอบหาจุดเกิดข้อมูลปลอม (Hallucination)
- สัปดาห์ที่ 4 - Reporting & Optimization: รวบรวมสรุปเอกสารผลการทดสอบ ดำเนินการแก้ไขจุดที่เป็นปัญหาขอขาดใน SQLAlchemy และอนุมัติปิดโปรเจกต์